DAY 03 – Functions and Lists

Name: M. Lalitha Sowjanya
Team: 4
Date: 11-06-26

---

1. OBJECTIVE:

To understand and apply functions and lists in Python through practical programming exercises.

---

2. TASKS ASSIGNED:

2.1. Functions

- Function is a block of reusable code to perform specific task.
- We use "def" keyword to define a function.

Why Functions?

- Reusable code
- Reduces code repetition
- Improves program structure

Syntax of a Function:

```
def function_name(parameters):
    # code
```

Components of a Function:

- **Function Definition:** Creating the function using "def" keyword.
- **Function Call:** Used to execute the function.
- **Parameters:** Values received by the function
- **Arguments:** Values passed to the function during the function call.

Types of Functions:

1. **Built-in Functions:**
   - These functions are also called as predefined functions provided by Python which can be used directly in programs.
   - **Examples:**
     - print()
     - input()
     - type()
     - len()
     - sum()

2. **User-defined Functions:**
   - Functions created by the programmer.
   - **Example:**

```
def display():
    print("This is a Function")
display()
```

Output:

This is a Function

2.2. Lists

- A List is a collection used to store multiple items in a single variable.
- It is denoted by square braces '[]'.

Features of a List:

- Can store multiple values.
- Lists can be modified (Mutable).
- Can store different data types.
- Allows duplicate values.

Syntax of a List:

```
List_name = ["item1", "item2", "item3"]
```

Common List Operations:

1. **append():**
   - Used to add an element at the end of the list.
2. **remove():**
   - Used to remove specific element from a list.
3. **len():**
   - Used to find the number of elements in a list.
4. **Accessing an element:**
   - Elements can be accessed using index.

Example:

```
students = ["Bala", "Lalitha"]
# Adding an element
students.append("Sowjanya")
print("After adding:", students)

# Removing an element
students.remove("Bala")
print("After removing:", students)

# Accessing an element
print("Accessing element at index 1:", students[1])
```

```
# Finding length
print("Length of list:", len(students))
```

Output:

```
After adding: ['Bala', 'Lalitha', 'Sowjanya']
After removing: ['Lalitha', 'Sowjanya']
Accessing element at index 1: Sowjanya
Length of list: 2
```

2.3. Programs

1. Create a Multiplication Table Generator using Functions

Code:

```
n = int(input("Enter a number to generate its multiplication table: "))
def Mul_Gen(n):
  for i in range(1, 11):
    print(n, "x", i, "=", n*i)
Mul_Gen(n)
```

Output:

```
Enter a number to generate its multiplication table: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Logic:

- Take a number as input from the user.
- Create a function to generate the multiplication table.
- Use a loop to iterate from 1 to 10.
- Multiply the entered number with each value in the loop.
- Display the multiplication table.
- Call the function to show the result.

2. Create a Simple Data Store using Lists

Code:

```
# Example: Grocery Items List
Grocery = []
```

```
n = int(input("How many grocery items you want to store: "))

for i in range(1, n+1):
  Entered_data = input(f"Enter grocery item {i}: ")
  Grocery.append(Entered_data)

print("\nGrocery items list: ", Grocery)
```

Output:

```
How many grocery items you want to store: 2
Enter grocery item 1: Oil
Enter grocery item 2: Rice Flour

Grocery items list:  ['Oil', 'Rice Flour']
```

Logic:

- Create an empty list to store grocery items.
- Ask the user how many grocery items they want to store.
- Use a loop to get each grocery item from the user.
- Add each item to the grocery list using append().
- Display all the grocery items stored in the list.

3. **Add Student Names into a List**

Code:

```
students = []
print("Before adding student names into the list: ", students)

students.append("Bala")
students.append("Lalitha")
students.append("Sowjanya")
print("After adding student names into the list: ", students)
```

Output:

```
Before adding student names into the list:  []
After adding student names into the list:  ['Bala', 'Lalitha', 'Sowjanya']
```

Logic:

- Create an empty list to store student names.
- Display the list before adding any names.
- Add student names to the list using append().
- Display the list after adding the student names.
- Verify that the names have been successfully added to the list.

4. **Display all Student Names using a Loop**

Code:

```
students = ["Bala", "Lalitha", "Sowjanya"]
```

```
i = 1
for student in students:
  print(i," ", student)
  i += 1
```

Output:

```
1  Bala
2  Lalitha
3  Sowjanya
```

Logic:

- Create a list containing student names.
- Initialize a variable i with the value 1.
- Use a loop to iterate through each student name in the list.
- Display the student number and the student name.
- Increment the value of i after each iteration.
- Continue until all student names are displayed.

5. **Create a Function to Add Numbers**

Code:

```
# Adding 2 numbers
num1 = int(input("Enter a number: "))
num2 = int(input("Enter another number: "))
def Add(num1, num2):
  print(num1 + num2)
Add(num1, num2)
```

Output:

```
Enter a number: 4
Enter another number: 3
7
```

Logic:

- Take two numbers as input from the user.
- Create a function to add the two numbers.
- Pass the entered numbers as arguments to the function.
- Calculate the sum of the two numbers inside the function.
- Display the result.
- Call the function to perform the addition.

6. **Create a Function to Display Student Details**

Code:

```
Students = ["Lalitha", "001", "CSM", "3rd Year"]

def Stu_Det(Students):
  print("--- Student Details ---\n")
  print("Name: ", Students[0])
```

```
    print("Roll No.: ", Students[1])
    print("Branch: ", Students[2])
    print("Year: ", Students[3])

Stu_Det(Students)
```

Output:

```
--- Student Details ---

Name:  Lalitha
Roll No.:  001
Branch:  CSM
Year:  3rd Year
```

Logic:

- Create a list to store student details such as name, roll number, branch, and year.
- Create a function to display the student details.
- Access each detail from the list using its index position.
- Display the student details with appropriate labels.
- Call the function and pass the student details list as an argument.
- Display all the student information on the screen.

---

3. LEARNINGS:

- Learned how to create and use functions in Python.
- Understood how to store and manage data using lists.
- Learned to add and access elements in a list.
- Used loops to display data from a list.
- Applied functions and lists to solve simple programming problems.